Input size

Outline

Input size

Asymptotic notation: a recall

Input instance consisting of one natural number

Input instance consisting of two natural numbers

Input instance consisting of an array of natural numbers
- An upper bound
- A lower bound
- Entries of constant size
- Two variants of the array input

Input size and number of steps

Why do we need a measure of the input?

- To compare two algorithms that solve the same problem we **count how many operations** they execute
- This number grows with the input:
  - sorting an array of one million entries takes longer than sorting an array of ten entries
  - testing whether a number with twenty digits is prime takes longer than testing a number with two digits
- To make this statement precise we need a measure of how *large* an input is

The plan of the chapter

- The **input size** is the number of bits needed to write the input in binary, as in the chapter «Representation of numbers»
- We define the input size and recall the asymptotic notation of the chapter «Asymptotic notation»
- We compute the input size, and its order of growth, for one natural number, two natural numbers, and an array of natural numbers
- We show why the distinction matters: the same number of steps can be small or huge with respect to the input size, depending on whether n is the *value* of a number or the *number of entries* of an array

Input size

Introduction: Input size

- We define the **input size** of an algorithm
- It is the measure with which we will express the running time of every algorithm of the course
- We recall how many bits a natural number needs, and we compute the input size of three small instances

Definition 1: input size

*The **input size** $|I|$ of an algorithm is the **total number of bits needed to represent a given input instance I in binary**.*

The input size as a function

- The input size is a function that maps each input instance to a natural number: the total number of bits needed to represent the input instance in binary
- We consider algorithms whose input instances consist of
  - one or more natural numbers, or
  - arrays of natural numbers

How many bits a natural number needs

- In the chapter «Representation of numbers» we have seen that the minimum number of digits needed to write a natural number $n \geq 1$ in base 2 is

$$\lfloor \log_2 n \rfloor + 1$$

- Equivalently, a natural number n is written with exactly k bits if and only if

$$2^{k-1} \leq n \leq 2^k - 1.$$

Example (1/3)

One natural number, $n = 37$.

- Since $2^5 = 32 \leq 37 < 64 = 2^6$, we have $\lfloor \log_2 37 \rfloor = 5$
- Hence

$$37 = (100101)_2, \qquad |I| = \lfloor \log_2 37 \rfloor + 1 = 6.$$

Example (2/3)

Two natural numbers, $n = 37$ and $m = 200$.

- Since $2^7 = 128 \leq 200 < 256 = 2^8$, the number $200 = (11001000)_2$ needs 8 bits
- The number 37 needs 6 bits, and

$$|I| = 6 + 8 = 14.$$

Example (3/3)

The array $a = [6, 1, 3, 8]$.

- Its entries are $6 = (110)_2$, $1 = (1)_2$, $3 = (11)_2$ and $8 = (1000)_2$
- Hence

$$|I| = 3 + 1 + 2 + 4 = 10.$$

From exact values to growth

The exact input size depends on the specific values of the instance.

In the analysis of algorithms we are interested in how it *grows*, and for this we use the **asymptotic notation**.

Summary

- The input size $|I|$ is the **total number of bits** needed to write the input instance in binary
- A natural number $n \geq 1$ needs $\lfloor \log_2 n \rfloor + 1$ bits, that is, exactly k bits if and only if $2^{k-1} \leq n \leq 2^k - 1$
- The input size of an instance is the sum of the bits of all its numbers: 6 for 37, 14 for 37 and 200, 10 for $[6, 1, 3, 8]$

Asymptotic notation: a recall

Introduction: Asymptotic notation: a recall

- The asymptotic notation is defined and studied in the chapter «Asymptotic notation»
- Here we only recall the definitions and the two facts that are used in this chapter
- They are the tools with which we describe how the input size grows

The setting

- As in the chapter «Asymptotic notation», f and g are functions
  - defined on the natural numbers $n \geq 1$
  - with non-negative real values
- The notation compares their **orders of growth** for all sufficiently large n

The asymptotic notation (1/2)

- $f(n) = O\big(g(n)\big)$ (**upper bound**) if there exist $c \in \mathbb{R}_{>0}$ and $n_1 \in \mathbb{N}$ such that

$$0 \leq f(n) \leq c\, g(n), \qquad \forall n \geq n_1;$$

- $f(n) = \Omega\big(g(n)\big)$ (**lower bound**) if there exist $c \in \mathbb{R}_{>0}$ and $n_1 \in \mathbb{N}$ such that

$$0 \leq c\, g(n) \leq f(n), \qquad \forall n \geq n_1;$$

The asymptotic notation (2/2)

- $f(n) = \Theta\big(g(n)\big)$ (**both**) if there exist $c_1, c_2 \in \mathbb{R}_{>0}$ and $n_1 \in \mathbb{N}$ such that

$$0 \leq c_1\, g(n) \leq f(n) \leq c_2\, g(n), \qquad \forall n \geq n_1.$$

How we use the definitions

- To prove one of these statements we **determine** the constants (c, or c_1 and c_2) and the threshold n_1
- For functions of two parameters, such as $f(n, m)$, the definitions are the same, with the inequalities required for every $n \geq n_1$ and every $m \geq m_1$
- The following two facts are proved in the chapter «Asymptotic notation»

Observation 1: Θ is the same as O and Ω

We have $f(n) = \Theta\big(g(n)\big)$ *if and only if* $f(n) = O\big(g(n)\big)$ *and* $f(n) = \Omega\big(g(n)\big)$.

Proposition 1: limit criterion

Let $g(n) > 0$ *for every* $n \geq 1$. *If*

$$\lim_{n \to \infty} \frac{f(n)}{g(n)} = L \in \mathbb{R}_{>0},$$

then $f(n) = \Theta\big(g(n)\big)$.

Summary

- O is an **upper bound**, Ω a **lower bound**, Θ both, for all sufficiently large n
- To prove them we determine the constants and the threshold; with two parameters, a threshold for each
- Θ holds if and only if both O and Ω hold
- A positive finite limit of $f(n)/g(n)$ gives $f(n) = \Theta\big(g(n)\big)$

Input instance consisting of one natural number

Introduction: one natural number

- We start from the simplest input: a **single natural number**
- We prove that its input size grows as $\log_2 n$
- This is the first example of a proof with the constants c_1, c_2 and the threshold n_1

Input size of one natural number

- If the input consists of a single natural number $n \geq 1$, then the input size is

$$|I| = \lfloor \log_2 n \rfloor + 1,$$

  which is the minimum number of bits needed to represent n in binary

Observation 2: input size of one natural number

We have

$$\lfloor \log_2 n \rfloor + 1 = \Theta(\log_2 n).$$

Proof (1/2)

To prove the result, we determine $c_1, c_2 \in \mathbb{R}_{>0}$ and $n_1 \in \mathbb{N}$ such that

$$0 \leq c_1 \log_2 n \leq \lfloor \log_2 n \rfloor + 1 \leq c_2 \log_2 n, \qquad \forall n \geq n_1.$$

Since $x < \lfloor x \rfloor + 1$ for every real number x, we have

$$\log_2 n \leq \lfloor \log_2 n \rfloor + 1, \qquad \forall n \geq 1.$$

Proof (2/2)

Moreover, since $\lfloor x \rfloor \leq x$ for every real number x, and $\log_2 n \geq 1$ for every $n \geq 2$, we have

$$\lfloor \log_2 n \rfloor + 1 \leq \log_2 n + 1 \leq 2 \log_2 n, \qquad \forall n \geq 2.$$

Thus, by choosing $c_1 = 1$, $c_2 = 2$, and $n_1 = 2$, we obtain

$$0 \leq \log_2 n \leq \lfloor \log_2 n \rfloor + 1 \leq 2 \log_2 n, \qquad \forall n \geq 2.$$

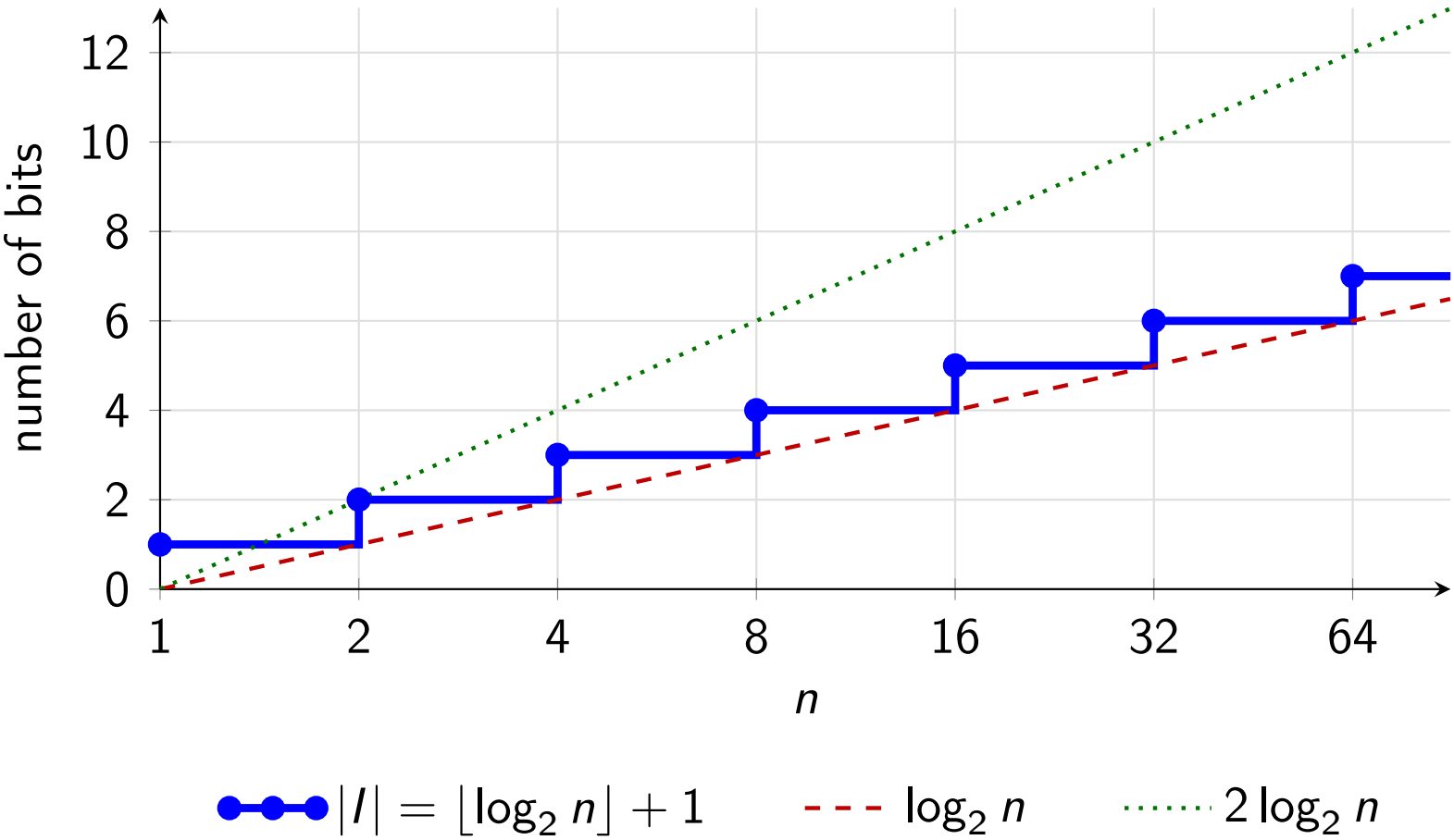
□

One natural number: the order of growth

If the input of an algorithm consists of
a single natural number $n \geq 1$,
then the input size satisfies:

$$|I| = \Theta(\log_2 n).$$

The input size of one natural number



Reading the graph

- The graph shows the input size $|I| = \lfloor \log_2 n \rfloor + 1$ of one natural number n, with the horizontal axis in **logarithmic scale**
- It increases by **one bit at every power of** 2
- For $n \geq 2$ it lies between $\log_2 n$ and $2 \log_2 n$, as stated by the observation that we have just proved

Example

- For $n = 37$ we have $|I| = 6$ (first example of the chapter) and $\log_2 37 \approx 5.21$
- The inequalities of the proof become

$$\log_2 37 \approx 5.21 \ \leq \ |I| = 6 \ \leq \ 2 \log_2 37 \approx 10.42.$$

- For $n = 1$ we have $|I| = 1$ but $\log_2 1 = 0$: this is why the threshold of the proof is $n_1 = 2$

Summary

- One natural number $n \geq 1$: $|I| = \lfloor \log_2 n \rfloor + 1$
- $|I| = \Theta(\log_2 n)$, with $c_1 = 1$, $c_2 = 2$ and $n_1 = 2$:

$$\log_2 n \leq \lfloor \log_2 n \rfloor + 1 \leq 2 \log_2 n, \qquad \forall n \geq 2$$

- The threshold $n_1 = 2$ is needed because $\log_2 1 = 0$

Input instance consisting of two natural numbers

Introduction: two natural numbers

- When the input consists of two numbers, the bits of **both** have to be written
- We prove that the input size grows as $\log_2 n + \log_2 m$
- and also as the logarithm of the **larger** of the two numbers

Input size of two natural numbers

- If the input consists of two natural numbers $n, m \geq 1$, then the input size is

$$|I| = \big(\lfloor \log_2 n \rfloor + 1\big) + \big(\lfloor \log_2 m \rfloor + 1\big),$$

  which is the minimum number of bits needed to represent both n and m in binary

Observation 3: input size of two natural numbers

We have

$$\left(\lfloor \log_2 n \rfloor + 1\right) + \left(\lfloor \log_2 m \rfloor + 1\right) = \Theta(\log_2 n + \log_2 m).$$

Proof (1/4)

To prove the result, we determine $c_1, c_2 \in \mathbb{R}_{>0}$ and $n_1, m_1 \in \mathbb{N}$ such that

$$\begin{aligned} 0 \leq c_1(\log_2 n + \log_2 m) &\leq \big(\lfloor \log_2 n \rfloor + 1\big) + \big(\lfloor \log_2 m \rfloor + 1\big) \\ &\leq c_2(\log_2 n + \log_2 m) \end{aligned}$$

for every $n \geq n_1$ and $m \geq m_1$.

Proof (2/4)

We have

$$\log_2 n \leq \lfloor \log_2 n \rfloor + 1, \qquad \log_2 m \leq \lfloor \log_2 m \rfloor + 1, \qquad \forall n, m \geq 1.$$

Therefore,

$$\log_2 n + \log_2 m \leq \big(\lfloor \log_2 n \rfloor + 1\big) + \big(\lfloor \log_2 m \rfloor + 1\big), \qquad \forall n, m \geq 1.$$

Proof (3/4)

Moreover,

$$\lfloor \log_2 n \rfloor + 1 \leq \log_2 n + 1, \qquad \lfloor \log_2 m \rfloor + 1 \leq \log_2 m + 1.$$

Hence

$$\big(\lfloor \log_2 n \rfloor + 1\big) + \big(\lfloor \log_2 m \rfloor + 1\big) \leq (\log_2 n + 1) + (\log_2 m + 1).$$

If $n, m \geq 2$, then $\log_2 n \geq 1$ and $\log_2 m \geq 1$, so

$$\log_2 n + 1 \leq 2 \log_2 n, \qquad \log_2 m + 1 \leq 2 \log_2 m.$$

Proof (4/4)

Thus,

$$\left(\lfloor \log_2 n \rfloor + 1\right) + \left(\lfloor \log_2 m \rfloor + 1\right) \leq 2(\log_2 n + \log_2 m), \qquad \forall n, m \geq 2.$$

Therefore, by choosing $c_1 = 1$, $c_2 = 2$, $n_1 = 2$, and $m_1 = 2$, we obtain

$$\begin{aligned} 0 \leq \log_2 n + \log_2 m &\leq \left(\lfloor \log_2 n \rfloor + 1\right) + \left(\lfloor \log_2 m \rfloor + 1\right) \\ &\leq 2(\log_2 n + \log_2 m), \qquad \forall n, m \geq 2. \end{aligned}$$

□

Two natural numbers: the order of growth

If the input of an algorithm consists of
two natural numbers $n, m \geq 1$,
then the input size satisfies:

$$|I| = \Theta(\underbrace{\log_2 n + \log_2 m}_{\log_2(nm)}).$$

The larger of the two numbers

The sum of the two logarithms can be replaced
by the logarithm of the **larger** of the two numbers:

the larger number needs at least half of the bits of the instance.

Observation 4: input size of two natural numbers and their maximum

We have

$$\left(\lfloor \log_2 n \rfloor + 1\right) + \left(\lfloor \log_2 m \rfloor + 1\right) = \Theta(\log_2 M),$$

where

$$M = \max\{n, m\}.$$

Proof (1/4)

To prove the result, we determine $c_1, c_2 \in \mathbb{R}_{>0}$ and $n_1, m_1 \in \mathbb{N}$ such that

$$\begin{aligned} 0 \leq c_1 \log_2 M &\leq \big(\lfloor \log_2 n \rfloor + 1\big) + \big(\lfloor \log_2 m \rfloor + 1\big) \\ &\leq c_2 \log_2 M, \qquad \forall n \geq n_1,\ m \geq m_1. \end{aligned}$$

Since one of the two numbers is equal to M, we have

$$\lfloor \log_2 M \rfloor + 1 \leq \big(\lfloor \log_2 n \rfloor + 1\big) + \big(\lfloor \log_2 m \rfloor + 1\big).$$

Proof (2/4)

Moreover,

$$\log_2 M \leq \lfloor \log_2 M \rfloor + 1, \qquad \forall M \geq 1.$$

Therefore,

$$\log_2 M \leq \big(\lfloor \log_2 n \rfloor + 1\big) + \big(\lfloor \log_2 m \rfloor + 1\big), \qquad \forall n, m \geq 1.$$

Proof (3/4)

On the other hand, since $n \leq M$ and $m \leq M$, we have

$$\lfloor \log_2 n \rfloor + 1 \leq \lfloor \log_2 M \rfloor + 1, \qquad \lfloor \log_2 m \rfloor + 1 \leq \lfloor \log_2 M \rfloor + 1.$$

Hence

$$\big(\lfloor \log_2 n \rfloor + 1\big) + \big(\lfloor \log_2 m \rfloor + 1\big) \leq 2\big(\lfloor \log_2 M \rfloor + 1\big).$$

If $M \geq 2$, then $\log_2 M \geq 1$, so

$$\lfloor \log_2 M \rfloor + 1 \leq \log_2 M + 1 \leq 2 \log_2 M.$$

Proof (4/4)

Thus,

$$\left(\lfloor \log_2 n \rfloor + 1\right) + \left(\lfloor \log_2 m \rfloor + 1\right) \leq 4 \log_2 M, \qquad \forall M \geq 2.$$

Therefore, by choosing $c_1 = 1$, $c_2 = 4$, $n_1 = 2$, and $m_1 = 2$ (which give $M \geq 2$), we obtain

$$\begin{aligned} 0 \leq \log_2 M &\leq \left(\lfloor \log_2 n \rfloor + 1\right) + \left(\lfloor \log_2 m \rfloor + 1\right) \\ &\leq 4 \log_2 M, \qquad \forall n, m \geq 2. \end{aligned}$$

□

Two natural numbers and their maximum

If the input of an algorithm consists of
two natural numbers $n, m \geq 1$,
then the input size satisfies:

$$|I| = \Theta\big(\log_2 M\big),$$

where

$$M = \max\{n, m\}.$$

Example

- For $n = 37$ and $m = 200$ we have $|I| = 14$ (first example of the chapter), $M = 200$, and

$$\log_2 37 + \log_2 200 \approx 12.85, \qquad \log_2 200 \approx 7.64.$$

- The inequalities of the proofs of the two observations become

$$12.85 \leq 14 \leq 2 \cdot 12.85 = 25.71$$

and

$$7.64 \leq 14 \leq 4 \cdot 7.64 = 30.58,$$

where all the values are rounded to two decimals

Summary

- Two natural numbers $n, m \geq 1$:
  $|I| = \left(\lfloor \log_2 n \rfloor + 1\right) + \left(\lfloor \log_2 m \rfloor + 1\right)$
- $|I| = \Theta(\log_2 n + \log_2 m)$, with $c_1 = 1$, $c_2 = 2$, $n_1 = m_1 = 2$
- $|I| = \Theta(\log_2 M)$ with $M = \max\{n, m\}$, with $c_1 = 1$, $c_2 = 4$, $n_1 = m_1 = 2$: the larger number needs at least half of the bits

Input instance consisting of an array of natural numbers

Introduction: an array of natural numbers

- Most of the algorithms of the course receive an **array**
- Its input size depends both on the number n of entries and on their values
- We prove an upper bound and a lower bound, we see when they meet, and we look at two variants of the array input

Input size of an array

- If the input instance consists of an array $a[1..n]$ of n natural numbers, with $a[i] \geq 1$ for every $i \in \{1, 2, \ldots, n\}$, then the input size is

$$|I| = \sum_{i=1}^{n} \left(\lfloor \log_2 a[i] \rfloor + 1 \right),$$

  which is the minimum number of bits needed to represent all the entries of the array in binary

Observation 5: upper bound on the input size of an array

Given an array $a[1..n]$ *of* n *natural numbers, with* $a[i] \geq 1$ *for every* $i \in \{1, 2, \ldots, n\}$*, we have*

$$\sum_{i=1}^{n} \left(\lfloor \log_2 a[i] \rfloor + 1 \right) = O\big(n(1 + \log_2 M)\big),$$

where

$$M = \max\big\{a[1], a[2], \ldots, a[n]\big\}.$$

Proof (1/2)

To prove the result, we determine $c \in \mathbb{R}_{>0}$ and $n_1 \in \mathbb{N}$ such that

$$0 \leq \sum_{i=1}^{n} \big(\lfloor \log_2 a[i] \rfloor + 1 \big) \leq c\, n \big(1 + \log_2 M \big), \qquad \forall n \geq n_1.$$

Since $a[i] \leq M$ for every $i \in \{1, 2, \ldots, n\}$, we have

$$\lfloor \log_2 a[i] \rfloor + 1 \leq \lfloor \log_2 M \rfloor + 1, \qquad \forall i \in \{1, 2, \ldots, n\}.$$

Therefore,

$$\sum_{i=1}^{n} \big(\lfloor \log_2 a[i] \rfloor + 1 \big) \leq \sum_{i=1}^{n} \big(\lfloor \log_2 M \rfloor + 1 \big) = n \big(\lfloor \log_2 M \rfloor + 1 \big).$$

Proof (2/2)

Moreover, we have

$$\lfloor \log_2 M \rfloor + 1 \leq \log_2 M + 1.$$

Hence,

$$\sum_{i=1}^{n} \big(\lfloor \log_2 a[i] \rfloor + 1 \big) \leq n\big(1 + \log_2 M\big), \qquad \forall n \geq 1.$$

Therefore, by choosing $c = 1$ and $n_1 = 1$, we obtain

$$0 \leq \sum_{i=1}^{n} \big(\lfloor \log_2 a[i] \rfloor + 1 \big) \leq n\big(1 + \log_2 M\big), \qquad \forall n \geq 1.$$

□

An array: the upper bound

If the input of an algorithm consists of an array $a[1..n]$ of n natural numbers, with $a[i] \geq 1$ for every $i \in \{1, 2, \ldots, n\}$, then the input size satisfies

$$|I| = O\big(n(1 + \log_2 M)\big),$$

where

$$M = \max\big\{a[1], a[2], \ldots, a[n]\big\}.$$

Is the upper bound tight?

- The upper bound $n(1 + \log_2 M)$ counts every entry **as if it were as large as the maximum**
- When only a few entries are large, this is a strong overestimate
- The bound cannot be replaced by Θ: we show it with a counterexample

Example: the upper bound is not tight (1/4)

Given an array $a[1..n]$ of n natural numbers, with $a[i] \geq 1$ for every $i \in \{1, 2, \ldots, n\}$, to prove that, in general,

$$\sum_{i=1}^{n} \left(\lfloor \log_2 a[i] \rfloor + 1 \right) \neq \Theta\big(n(1 + \log_2 M)\big),$$

where

$$M = \max\{a[1], a[2], \ldots, a[n]\},$$

it is enough to provide a **counterexample**.

- Consider the family of arrays defined by

$$a[1] = M, \qquad a[2] = a[3] = \cdots = a[n] = 1,$$

with $M \geq 2$.

Example: the upper bound is not tight (2/4)

- For these arrays, the input size is

$$|I| = \sum_{i=1}^{n} \left(\lfloor \log_2 a[i] \rfloor + 1 \right)$$

$$= (\lfloor \log_2 M \rfloor + 1) + (n-1)(\lfloor \log_2 1 \rfloor + 1).$$

- Since $\lfloor \log_2 1 \rfloor + 1 = 1$, we obtain

$$|I| = (\lfloor \log_2 M \rfloor + 1) + (n-1) = n + \lfloor \log_2 M \rfloor.$$

Example: the upper bound is not tight (3/4)

- Now choose $M = 2^n$, so that $\log_2 M = n$
- Then

$$|I| = n + n = 2n,$$

whereas

$$n(1 + \log_2 M) = n(1 + n) = n^2 + n.$$

Example: the upper bound is not tight (4/4)

- If we had $2n = \Theta(n^2 + n)$, there would exist $c_1 \in \mathbb{R}_{>0}$ and $n_1 \in \mathbb{N}$ such that

$$c_1(n^2 + n) \leq 2n, \qquad \forall n \geq n_1,$$

that is,

$$c_1(n + 1) \leq 2, \qquad \forall n \geq n_1,$$

which is false for n large enough

Thus, for this family of arrays, $|I| \neq \Theta\big(n(1 + \log_2 M)\big)$.

Two special cases of the upper bound

- If $M \geq 2$, then $\log_2 M \geq 1$, and therefore $1 + \log_2 M \leq 2 \log_2 M$. Hence

$$n(1 + \log_2 M) \leq 2n \log_2 M,$$

and the input size satisfies

$$|I| = O\big(n \log_2 M\big).$$

- If $M = 1$, then all the entries of the array are equal to 1, and therefore

$$|I| = \sum_{i=1}^{n} (\lfloor \log_2 1 \rfloor + 1) = n.$$

Towards a lower bound

Every entry of the array needs **at least one bit**.

This simple remark gives a lower bound
that holds for every array.

Observation 6: lower bound on the input size of an array

Given an array $a[1..n]$ *of* n *natural numbers, with* $a[i] \geq 1$ *for every* $i \in \{1, 2, \ldots, n\}$*, we have*

$$\sum_{i=1}^{n} \left(\lfloor \log_2 a[i] \rfloor + 1 \right) = \Omega(n).$$

Proof (1/2)

To prove the result, we determine $c \in \mathbb{R}_{>0}$ and $n_1 \in \mathbb{N}$ such that

$$0 \leq c\, n \leq \sum_{i=1}^{n} \big(\lfloor \log_2 a[i] \rfloor + 1 \big), \qquad \forall n \geq n_1.$$

Since $a[i] \geq 1$ for every $i \in \{1, 2, \ldots, n\}$, we have

$$\lfloor \log_2 a[i] \rfloor + 1 \geq \lfloor \log_2 1 \rfloor + 1 = 1, \qquad \forall i \in \{1, 2, \ldots, n\}.$$

Therefore,

$$\sum_{i=1}^{n} \big(\lfloor \log_2 a[i] \rfloor + 1 \big) \geq \sum_{i=1}^{n} 1 = n.$$

Proof (2/2)

Thus, by choosing $c = 1$ and $n_1 = 1$, we obtain

$$0 \leq n \leq \sum_{i=1}^{n} \left(\lfloor \log_2 a[i] \rfloor + 1 \right), \qquad \forall n \geq 1.$$

□

An array: the lower bound

If the input of an algorithm consists of an array $a[1..n]$ of n natural numbers, with $a[i] \geq 1$ for every $i \in \{1, 2, \ldots, n\}$, then the input size satisfies

$$|I| = \Omega(n).$$

Example

- For the array $a = [6, 1, 3, 8]$ of the first example of the chapter we have

$$n = 4, \qquad |I| = 10, \qquad M = 8$$

- The lower bound and the upper bound of the two observations give

$$n = 4 \ \leq \ |I| = 10 \ \leq \ n(1 + \log_2 M) = 4\,(1 + 3) = 16.$$

When the two bounds meet

In many applications the entries of the array
are stored with a **fixed number of bits**, independent of n.

In this case the two bounds meet.

Corollary 1: entries of constant size

Given an array $a[1..n]$ *of* n *natural numbers, with* $a[i] \geq 1$ *for every* $i \in \{1, 2, \ldots, n\}$*, if all the entries of the array can be represented using at most a fixed constant number of bits, then*

$$|I| = \Theta(n).$$

Proof

By hypothesis there exists a constant $c \in \mathbb{N}$ such that

$$\lfloor \log_2 a[i] \rfloor + 1 \leq c, \qquad \forall i \in \{1, 2, \ldots, n\}.$$

Therefore,

$$|I| = \sum_{i=1}^{n} \left(\lfloor \log_2 a[i] \rfloor + 1 \right) \leq \sum_{i=1}^{n} c = cn, \qquad \forall n \geq 1,$$

and hence $|I| = O(n)$.

By the lower bound on the input size of an array we also have $|I| = \Omega(n)$, and therefore, since Θ is the same as O and Ω,

$$|I| = \Theta(n).$$



Two variants of the array input

- The bounds above extend to two variants of the array input:
  - the input contains also the **number** n of entries
  - the entries of the array are allowed to be 0
- In both cases the same orders of growth hold

The number n and the array (1/3)

- If the input consists of a natural number $n \geq 1$ and an array $a[1..n]$ of n natural numbers, with $a[i] \geq 1$ for every $i \in \{1, 2, \ldots, n\}$, then the input size is

$$|I| = \left(\lfloor \log_2 n \rfloor + 1\right) + \sum_{i=1}^{n} \left(\lfloor \log_2 a[i] \rfloor + 1\right).$$

- Let

$$M = \max\{n, a[1], a[2], \ldots, a[n]\}.$$

The number n and the array (2/3)

Then we have

$$\lfloor \log_2 n \rfloor + 1 \leq 1 + \log_2 M,$$

and, for every $i \in \{1, 2, \ldots, n\}$,

$$\lfloor \log_2 a[i] \rfloor + 1 \leq 1 + \log_2 M.$$

Therefore,

$$|I| \leq (1 + \log_2 M) + n(1 + \log_2 M) = (n + 1)(1 + \log_2 M).$$

Hence

$$|I| = O\big((n + 1)(1 + \log_2 M)\big).$$

The number n and the array (3/3)

- Since $n + 1 \leq 2n$ for every $n \geq 1$, we also obtain

$$|I| = O\big(n(1 + \log_2 M)\big).$$

- Moreover, if $M \geq 2$, then $1 + \log_2 M \leq 2 \log_2 M$, and therefore

$$|I| = O\big(n \log_2 M\big).$$

- The lower bound $|I| = \Omega(n)$ still holds, since the bits of the array alone are at least n

Entries equal to 0 (1/3)

- If the entries of the array are allowed to be 0, then the input size is

$$|I| = \sum_{i=1}^{n} \left(\lfloor \log_2(\max\{1, a[i]\}) \rfloor + 1 \right),$$

so that each entry equal to 0 contributes exactly 1 bit

Entries equal to 0 (2/3)

- In this case, if we let

$$M = \max\{1, a[1], a[2], \ldots, a[n]\},$$

then the previous formulas become

$$|I| = O\big(n(1 + \log_2 M)\big),$$

and, if $M \geq 2$,

$$|I| = O(n \log_2 M).$$

Entries equal to 0 (3/3)

- Moreover, we still have

$$|I| = \Omega(n),$$

since every entry contributes at least 1 bit

- In particular, if all the entries of the array can be represented using at most a fixed constant number of bits, then

$$|I| = \Theta(n).$$

Summary (1/2)

Array $a[1..n]$ with entries $a[i] \geq 1$:

- $|I| = \sum_{i=1}^{n} \left(\lfloor \log_2 a[i] \rfloor + 1 \right)$
- Upper bound: $|I| = O\big(n(1 + \log_2 M)\big)$, with M the maximum entry, and $|I| = O(n \log_2 M)$ if $M \geq 2$
- The upper bound is **not tight** in general: $a[1] = 2^n$ and all the other entries equal to 1 give $|I| = 2n$

Lower bound:

- $|I| = \Omega(n)$, because every entry needs at least one bit

Summary (2/2)

Entries of constant size:

- If every entry has at most a constant number of bits, $|I| = \Theta(n)$

Two variants:

- With the number n in the input, and $M = \max\{n, a[1], \ldots, a[n]\}$, the same bounds $O\big(n(1 + \log_2 M)\big)$ and $\Omega(n)$ hold
- With entries equal to 0, each of them counts 1 bit, and the same bounds hold with $M = \max\{1, a[1], \ldots, a[n]\}$

Input size and number of steps

Introduction: Input size and number of steps

- We can now explain **why the input size is measured in bits**
- We define the worst-case running time, and polynomial and exponential algorithms
- We see that the same number of steps can be polynomial or exponential, depending on the role of the parameter n

Counting steps

- The time needed by an algorithm is estimated by counting its **steps**:
  - in pseudo-code, a step is the execution of one line
  - the condition of a **while** loop counts as one step each time it is evaluated

- The number of steps is
  - first computed as a function $S(n)$ of a natural parameter n of the input
  - then expressed as a function of the input size $|I|$

Definition 2: worst-case running time

*The **worst-case running time** $T(|I|)$ of an algorithm is the maximum number of steps executed by the algorithm on any input I of a given input size $|I|$.*

Definition 3: polynomial and exponential algorithms

*An algorithm is **polynomial** if its worst-case running time satisfies* $T(|I|) = O(|I|^k)$ *for some constant* $k \in \mathbb{N}$.

*An algorithm is **exponential** if its worst-case running time satisfies* $T(|I|) = 2^{\Theta(|I|)}$, *that is, if there exist* $c_1, c_2 \in \mathbb{R}_{>0}$ *and* $k_1 \in \mathbb{N}$ *such that*

$$2^{c_1|I|} \leq T(|I|) \leq 2^{c_2|I|}, \qquad \forall |I| \geq k_1.$$

Two roles of the parameter n

- The parameter n can play two very different roles:
  - the **value** of a number given as input
  - the **number of entries** of an array given as input
- The next observation shows how n is related to $|I|$ in the two cases

Observation 7: value of a number and number of entries of an array

1. *If the input consists of a single natural number $n \geq 1$, then*

$$2^{|I|-1} \leq n \leq 2^{|I|} - 1 :$$

   the value n is exponential in the input size.

2. *If the input consists of an array $a[1..n]$ of n natural numbers (and possibly of the number n), then*

$$n \leq |I| :$$

   the number of entries n is at most linear in the input size.

Proof

1. Since $|I| = \lfloor \log_2 n \rfloor + 1$, we have $\lfloor \log_2 n \rfloor = |I| - 1$, that is,

$$|I| - 1 \leq \log_2 n < |I|.$$

   The function $x \mapsto 2^x$ is strictly increasing, so $2^{|I|-1} \leq n < 2^{|I|}$. Since n is an integer, $n \leq 2^{|I|} - 1$.

2. Every entry of the array contributes at least one bit to the input size (lower bound on the input size of an array, and the same holds for entries equal to 0), so $|I| \geq n$.

□

Constant factors

When a number of steps is bounded by exponentials with constant factors, the constant factors can be **absorbed in the exponent**.

Observation 8: constant factors in the exponent

Let $a_1, a_2, b_1, b_2 \in \mathbb{R}_{>0}$. If a function T satisfies

$$a_1\, 2^{b_1 x} \leq T(x) \leq a_2\, 2^{b_2 x}$$

for every sufficiently large natural number x, then $T(x) = 2^{\Theta(x)}$.

Proof (1/2)

We write the constant factors as powers of 2:

$$a_1 = 2^{\log_2 a_1}, \qquad a_2 = 2^{\log_2 a_2}.$$

Then

$$a_2\, 2^{b_2 x} = 2^{b_2 x + \log_2 a_2} \leq 2^{2 b_2 x} \qquad \text{whenever} \qquad b_2 x \geq \log_2 a_2,$$

and

$$a_1\, 2^{b_1 x} = 2^{b_1 x + \log_2 a_1} \geq 2^{\frac{b_1}{2} x} \qquad \text{whenever} \qquad \frac{b_1}{2}\, x \geq -\log_2 a_1.$$

Proof (2/2)

- Both conditions hold for every sufficiently large x
- Therefore, with $c_1 = \frac{b_1}{2}$ and $c_2 = 2b_2$, we obtain

$$2^{c_1 x} \leq T(x) \leq 2^{c_2 x}$$

for every sufficiently large x, that is, $T(x) = 2^{\Theta(x)}$

□

Example: the sum of the entries of an array (1/5)

- Consider the algorithm `SumArray` on the next slide, which computes the sum of the entries of an array
- We count its steps as a function of the number n of entries
- Then we express them as a function of the input size $|I|$

Example: the sum of the entries of an array (2/5)

Algorithm: SumArray

Input : An array $a[1..n]$ of $n \geq 1$ natural numbers
Output : $a[1] + a[2] + \cdots + a[n]$

```
1 function SumArray(a, n)
2     s := 0
3     i := 1
4     while i ≤ n do
5         s ← s + a[i]
6         i ← i + 1
7     end
8     return s
9 end
```

Run in Colab: Python · C++ · Java

Example: the sum of the entries of an array (3/5)

Step 1: number of steps.

- Lines 2, 3 and 8 are executed 1 time each
- The condition at line 4 is evaluated $n + 1$ times (it is false the last time)
- Lines 5 and 6 are executed n times each

Therefore

$$S(n) = 3 + (n + 1) + 2n = 3n + 4,$$

for every array of n entries, whatever its values.

Example: the sum of the entries of an array (4/5)

Step 2: from n to $|I|$.

- Every input of size $|I|$ has $n \leq |I|$ entries (number of entries of an array, in the observation above), so it requires at most $3|I| + 4$ steps
- The array of $|I|$ entries all equal to 1 has input size $|I|$ and requires exactly $3|I| + 4$ steps

Hence

$$T(|I|) = 3|I| + 4 = O(|I|).$$

Example: the sum of the entries of an array (5/5)

Step 3: conclusion.

- `SumArray` is **polynomial** (linear) in the input size
- If instead the same loop ran from 1 to the *value* of a number n given as input, the $\Theta(n)$ steps would be **exponential** in the input size
- because n can be as large as $2^{|I|} - 1$ (value of a number, in the observation above)

Summary

- The worst-case running time $T(|I|)$ is the maximum number of steps on inputs of size $|I|$
- **Polynomial**: $T(|I|) = O(|I|^k)$; **exponential**: $T(|I|) = 2^{\Theta(|I|)}$, and constant factors can be absorbed in the exponent
- The **value** of a number is exponential in $|I|$; the **number of entries** of an array is at most $|I|$
- `SumArray` executes $3n + 4$ steps and is linear in the input size

Summary (1/2)

Key points of the chapter

- The input size $|I|$ is the number of bits needed to write the input instance in binary
- One natural number n: $|I| = \lfloor \log_2 n \rfloor + 1 = \Theta(\log_2 n)$
- Two natural numbers n, m:
  $|I| = \Theta(\log_2 n + \log_2 m) = \Theta(\log_2 \max\{n, m\})$

Summary (2/2)

- An array of n natural numbers: $|I| = \Omega(n)$ and $|I| = O\big(n(1 + \log_2 M)\big)$, with M the maximum entry; the upper bound is not tight in general. If the entries have a constant number of bits, $|I| = \Theta(n)$
- The same number of steps $S(n)$ can be polynomial or exponential in $|I|$: the value of a number is exponential in its input size, while the number of entries of an array is at most its input size